

CS509 Laboratory Repository

Repository Overview

This repository contains the implementation for **Assignment 03 – Minimum Spanning Tree (MST)** of the CS509 laboratory.

The assignment implements and compares two standard MST algorithms for weighted, undirected, connected graphs:

- Kruskal's Algorithm
- Prim's Algorithm

Both algorithms use the same graph input. Their calculated MST weights are compared with the expected result stored in the `outputs/` directory. The total MST weight produced by Kruskal and Prim must also be equal.

For CSR-based processing, the input adjacency list is converted to CSR before the algorithm is executed. The CSR conversion time is not included in the reported algorithm execution time.

Student / Pair Details

- **Student Name:** Rajan Jha
- **Entry Number:** 2026AIM1019
- **Assignment Mode:** Single

Language and Environment

- **Programming Language:** C++17
- **Compiler:** GNU g++
- **C++ Standard:** C++17
- **Compiler Flags:** `-std=c++17 -O2 -Wall -Wextra -pedantic`
- **Build System:** GNU Make
- **Operating System:** Ubuntu Linux
- **Architecture:** x86-64

Only the C++ standard library is used. No external libraries are required.

Directory Structure

```
assignment_03/
├── src/
│   ├── graph.h
│   ├── graph_io.cpp
│   ├── kruskal.h
│   ├── kruskal.cpp
│   ├── prim.h
│   ├── prim.cpp
│   ├── mst.h
│   └── mst.cpp
├── tests/
│   └── mst/
│       ├── test_01.txt
│       ├── test_02.txt
│       ├── test_03.txt
│       ├── test_04.txt
│       └── test_05.txt
├── outputs/
│   ├── kruskals/
│   │   ├── test_01.txt
│   │   ├── test_02.txt
│   │   ├── test_03.txt
│   │   ├── test_04.txt
│   │   └── test_05.txt
│   └── prims/
│       ├── test_01.txt
│       ├── test_02.txt
│       ├── test_03.txt
│       ├── test_04.txt
│       └── test_05.txt
├── driver/
│   └── main.cpp
├── Makefile
└── README.md
```

Source Files

File	Purpose
<code>src/graph.h</code>	Defines graph, CSR, edge and MST result structures.
<code>src/graph_io.cpp</code>	Reads the adjacency-list graph and converts it to CSR. It also extracts each undirected edge once for Kruskal.
<code>src/kruskal.cpp</code>	Implements Kruskal's algorithm and the DSU/Union-Find structure.
<code>src/prim.cpp</code>	Implements Prim's algorithm using a minimum priority queue.
<code>src/mst.cpp</code>	Contains the helper used to sort MST edges before printing.
<code>driver/main.cpp</code>	Common menu-driven wrapper for selecting an algorithm and test case.
<code>Makefile</code>	Compiles the complete program.

Common Wrapper: Build and Usage

Compilation

Run the following commands from the repository root:

```
make
```

The Makefile compiles:

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic \  
    driver/main.cpp src/graph_io.cpp src/kruskal.cpp \  
    src/prim.cpp src/mst.cpp -o mst_driver
```

To remove the executable:

```
make clean
```

Execution

```
./mst_driver
```

The common wrapper displays:

1. Kruskal's MST
2. Prim's MST
3. Run both and compare
0. Exit

The program remains in a `while` loop. After a test finishes, it returns to the menu so the executable does not need to be restarted.

Assignment 03 – Minimum Spanning Tree

Assignment Mode

Single

Objective

The objective is to implement two greedy Minimum Spanning Tree algorithms and verify that they produce the same minimum total weight for the same weighted, undirected, connected graph.

The two required algorithms are:

1. Kruskal's Algorithm
2. Prim's Algorithm

An MST connects all V vertices using exactly $V-1$ edges, contains no cycle, and has minimum possible total edge weight.

Algorithm / Approach

Kruskal's Algorithm

Kruskal's algorithm first obtains the undirected edges of the graph and sorts them by non-decreasing edge weight.

The edges are then processed in sorted order. An edge is added to the MST only if its endpoints are currently in different connected components. A Disjoint Set Union (DSU), also called Union-Find, is used to detect cycles efficiently.

The algorithm stops after $V-1$ edges have been selected.

Prim's Algorithm

Prim's algorithm starts from vertex 0 for reproducibility.

A minimum priority queue is used to select the cheapest edge connecting a vertex already in the MST to a vertex outside the MST. The process continues until all vertices have been included.

Prim uses the CSR representation of the graph.

Input Format

Each MST test case is stored in a separate text file using a weighted, undirected adjacency-list representation.

The format is:

```
V E
u0 degree neighbor1 weight1 neighbor2 weight2 ...
u1 degree neighbor1 weight1 neighbor2 weight2 ...
...
u(V-1) degree neighbor1 weight1 ...
```

Where:

- V is the number of vertices.
- E is the number of undirected edges.
- Vertices are numbered from 0 to $V-1$.
- Each undirected edge appears in both endpoint adjacency lists.
- The weight is the same in both adjacency lists.
- E counts every undirected edge once.
- The graph is connected.
- Edge weights are integers and may be positive, zero, or negative.

Example:

```
5 7
0 2 1 2 3 6
1 4 0 2 2 3 3 8 4 5
2 2 1 3 4 7
3 3 0 6 1 8 4 9
4 3 1 5 2 7 3 9
```

CSR Conversion

The graph is first read as an adjacency list. The helper function:

```
CSRGraph convertToCSR(const Graph& graph);
```

converts the adjacency-list representation to CSR.

The conversion is preprocessing and is **not included in the measured Prim execution time**.

The timing for Prim begins only after the CSR graph has been created.

Runtime Measurement

Only the selected MST algorithm is timed.

The following are excluded from the reported execution time:

- Reading the input file
- Constructing the adjacency list
- CSR conversion
- Printing the MST edges
- Reading the expected output
- Comparing the calculated and expected results

The measured interval starts immediately before the MST function is called and ends immediately after it returns.

Test Cases

The assignment requires graph sizes of 10, 100, 10,000, 50,000 and 100,000 vertices. The included graphs are sparse.

Test File	V	E	Input Type
test_01.txt	10	20	Weighted undirected adjacency list → CSR for Prim
test_02.txt	100	250	Weighted undirected adjacency list → CSR for Prim
test_03.txt	10,000	25,000	Weighted undirected adjacency list → CSR for Prim
test_04.txt	50,000	125,000	Weighted undirected adjacency list → CSR for Prim
test_05.txt	100,000	250,000	Weighted undirected adjacency list → CSR for Prim

The graphs use approximately $E = 2.5V$ edges and are connected.

Expected Output

The expected MST total weights stored in `outputs/` are:

Test File	Expected MST Weight
<code>test_01.txt</code>	2037
<code>test_02.txt</code>	27712
<code>test_03.txt</code>	2361751
<code>test_04.txt</code>	11706333
<code>test_05.txt</code>	23576746

The edge lists selected by Prim and Kruskal do not have to be identical because a graph can have multiple valid MSTs. The total MST weight must be the same.

Test Results

The following results were measured by running each algorithm separately on every required test case.

Mandatory Graph Result Table

Mode	Test File	Input Type	Input Size	Expected Output	Actual Output	Algorithm Time	Status
Single	<code>test_01.txt</code>	Adjacency list → CSR before Prim	V=10, E=20	2037	2037	Kruskal: 0.002834 ms; Prim: 0.002284 ms	Pass
Single	<code>test_02.txt</code>	Adjacency list → CSR before Prim	V=100, E=250	27712	27712	Kruskal: 0.013991 ms; Prim: 0.018036 ms	Pass
Single	<code>test_03.txt</code>	Adjacency list → CSR before Prim	V=10,000, E=25,000	2361751	2361751	Kruskal: 1.975876 ms; Prim: 4.188704 ms	Pass
Single	<code>test_04.txt</code>	Adjacency list → CSR before Prim	V=50,000, E=125,000	11706333	11706333	Kruskal: 9.931475 ms; Prim: 23.707371 ms	Pass
Single	<code>test_05.txt</code>	Adjacency list → CSR before Prim	V=100,000, E=250,000	23576746	23576746	Kruskal: 40.233858 ms; Prim: 51.590542 ms	Pass

The exact execution time can vary between runs depending on CPU load and system conditions.
The values above are the measured values from the test run used for this README.

MST Comparison Table

File	V	E	Exp. Wt.	Kruskal Wt.	Prim Wt.	Kruskal Time	Prim Time	Equal?	Status
test01.txt	10	20	2037	2037	2037	0.002834 ms	0.002284 ms	Yes	Pass
test02.txt	100	250	27712	27712	27712	0.013991 ms	0.018036 ms	Yes	Pass
test03.txt	10,000	25,000	2361751	2361751	2361751	1.975876 ms	4.188704 ms	Yes	Pass
test04.txt	50,000	125,000	11706333	11706333	11706333	9.931475 ms	23.707371 ms	Yes	Pass
test05.txt	100,000	250,000	23576746	23576746	23576746	40.233858 ms	51.590542 ms	Yes	Pass

Actual Terminal Output Format

For a single algorithm, the program prints the selected MST edges, total MST weight, execution time and expected-weight validation.

For example:

```
Kruskal's MST
-----
```



```
MST edges:
0 2 89
...

Total MST weight: 2037
Execution time: 0.002834 ms
Expected MST weight: 2037
Validation: PASS
```

When option 3 is selected, both algorithms are run on the same graph and the final comparison is printed:

```
Kruskal vs Prim: PASS
```

Complexity

Kruskal's Algorithm

Let V be the number of vertices and E be the number of undirected edges.

- Sorting edges: $O(E \log E)$
- DSU operations: approximately $O(E \alpha(V))$
- Overall: $O(E \log E)$
- Space: $O(V + E)$

Prim's Algorithm

Using a binary min-priority queue:

- Overall: $O(E \log V)$
- Space: $O(V + E)$

CSR Conversion

The adjacency-list-to-CSR conversion takes:

- Time: $O(V + E)$
- Space: $O(V + E)$

This preprocessing time is excluded from the reported algorithm timing.

Assumptions and Constraints

- The graph is weighted and undirected.
- The graph is connected.
- Vertex numbering is 0 to $V-1$.
- Every undirected edge occurs in both endpoint adjacency lists.
- The edge count E counts each undirected edge once.
- Edge weights are integers.
- Negative, zero and positive edge weights are allowed.
- The same input graph is used for both Kruskal and Prim.
- For a connected graph with $V > 1$, the MST contains exactly $V-1$ edges.

References

1. Princeton Algorithms, Section 4.3 – Minimum Spanning Trees:

<https://algs4.cs.princeton.edu/43mst/>

2. CS509 Laboratory Assignment 03 specification.

The Princeton Algorithms reference was used to understand the standard implementations and properties of Kruskal's and Prim's MST algorithms.

Final Result

All five required MST test cases pass for both algorithms.

- Kruskal produces the expected MST weight for every test case.
- Prim produces the expected MST weight for every test case.
- Kruskal and Prim produce equal total MST weights for every test case.
- CSR conversion is performed before Prim timing and is excluded from the reported algorithm execution time.